

A Study on the Trade-offs Between Ionic and React Native Frameworks in Mobile Application Development

Jacob Hellström
jahe0089@student.umu.se

Umeå University
Department of Applied Physics and Electronics
Umeå, Sweden

Abstract

This paper explores the differences in Ionic and React Native based on a developer perspective. The objective is to find key strengths and weaknesses of these two frameworks. This information is intended to help developers, that want to create a cross-platform mobile application, chose between these frameworks. In this study, a survey as well as an analysis of online articles was performed, to support the survey findings. The findings of this study show that React Native and Ionic differ a lot in how applications run and how they are rendered on a device. The survey and analysis indicate that Ionic is easier to learn than React Native, and Ionic has a simple way of packaging a mobile application as a web app. Meanwhile React Native has a much better native compatibility and integration in a mobile device. But the number of participants in the survey was too small to draw any statistically significant conclusions.

Keywords

Ionic, React Native, mobile application framework, cross-platform, application development, framework comparison

1 Introduction

Mobile applications are essential in our everyday life. However, developing and maintaining separate native applications for Android and iOS platforms can be resource-intensive and costly. Over the years cross-platform frameworks have become all the more popular when it comes to mobile application development. These frameworks offer developers the ability to write a single codebase that can run on multiple operating systems, significantly increasing development efficiency. Two of the more popular frameworks are Ionic and React Native, according to [14]. But what sets them apart? What are their advantages and disadvantages?

This study has performed an online survey on a non-probability sample group. This sample group consisted of different type of developers that had experience with Ionic and/or React Native. To support the findings of the survey an analysis of online articles on this topic has also been performed. This study aims to aid software developers when choosing between Ionic and React Native, and help to find out what projects each framework is best suited for.

1.1 Objective

The research objective of this paper is to identify key strengths and weaknesses of Ionic and React Native frameworks based on the experience of developers and relevant articles.

2 Background and Theory

In total, there are an estimated 4.8 billion smartphone users in the world. This means that for tech startups and tech businesses that build consumer products, it is a baseline expectation for these products to be usable on smartphones, and for there to be a mobile app for the product. Traditionally, developers would build separate native applications for iOS and Android, resulting in increased development time and maintenance costs.

2.1 Cross-platform Frameworks

To address this challenge, various cross-platform frameworks have emerged over the years, enabling developers to create applications that run on multiple platforms using a single codebase [8]. Among these frameworks, Ionic and React Native have become two of the most widely adopted solutions [14]. Ionic was developed by Drifty Co. in 2013 [9]. However the Ionic framework is currently owned by OutSystems since 2022 [18]. React Native was developed by Meta in 2015 and has also been maintaining it ever since. Ionic leverages web technologies such as HTML, CSS, and JavaScript to build hybrid mobile apps. In contrast, React allows developers to build native applications using JavaScript and React components. Understanding the fundamental differences and trade-offs between these two frameworks is crucial for developers that aim to choose the most suitable technology for their mobile cross-platform projects.

2.2 Ionic

Ionic is an open source UI toolkit for building mobile apps using the following web technologies HTML, CSS, and JavaScript. With integrations for popular frameworks like Angular, React, and Vue. The framework is used to build and deploy apps that work across multiple platforms, such as native iOS, Android, and the web as a Progressive Web App, all with one code base. Ionic emulates native app UI guidelines and uses native SDKs, bringing the UI standards and device features of native apps together with the power and flexibility of the open web. Ionic uses Cordova to deploy the application natively, or run it in the browser as a Progressive Web App, [10]. This setup for how Ionic runs a mobile application is shown in Figure 1.

According to [1] Cordova is a framework that wraps HTML5, CSS3 and JavaScript into a web app. Thus avoiding each mobile platforms native development language. Applications using Cordova execute within wrappers targeted to each unique platform, and rely on standards-compliant API bindings to access each device's sensors, data, and network status. The Cordova application itself is implemented as a web page.

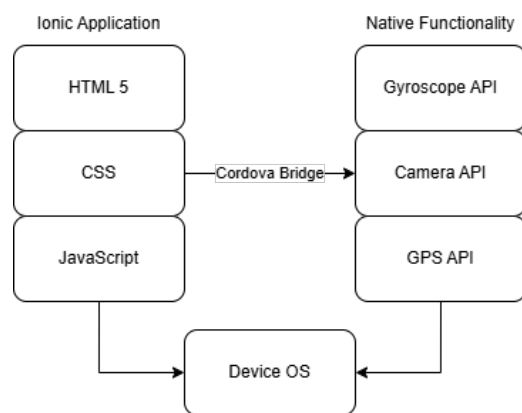


Figure 1: The architecture of Ionic. The diagram shows each component of an Ionic application, and how they are connected. With inspiration from [12].

Some popular apps built with the Ionic framework are Target, NBC News, and Panda Express, according to [8].

2.3 React Native

React Native is an open source UI toolkit, like Ionic. The framework is used for building Android and iOS applications using React and the application platforms native capabilities. It is written in JavaScript and rendered with native code. React render to the native platform UI, meaning an application uses the same native platform APIs that other apps do [5]. JavaScript is used to access the platforms APIs as well as to describe the appearance and behaviour of the UI using React components (bundles of reusable and nestable code). React Native runs on React, a common open source library for building user interfaces with JavaScript [15]. Developers can use React Native without a web framework like Angular or Vue. However it is recommended that you use a framework when developing in React Native. According to [4] the recommended framework is Expo. Which, according to [2], has been tailored for React Native for a very long time.

In Figure 2 the Bridge is what connects the JavaScript realm and the Native realm in a mobile application. These two realms run simultaneously in a React Native application. These are called realms to reference the fact that they run on different threads on the device. The Native realm is coded by Swift for iOS and Java for Android. The bridge can connect these realms into Shadow Tree, JSON, and Native modules. The authors of [5] describe React Native structure in Figure 2 as four core sections. Which are the following:

- The React code which the developer writes.
- The code written by the developer eventually interprets the JavaScript.
- A series of elements which are known as The Bridge collectively.
- The Native side.

The React Glossary [3] explains what a React Shadow Tree is. At its' core it is multiple Shadow Nodes that have been created by the Fabric Renderer. These nodes are objects that represent a React

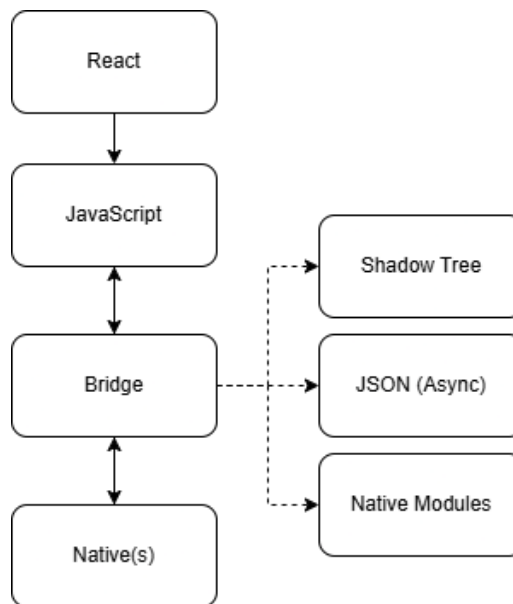


Figure 2: The architecture of React Native. The diagram shows each component of a React Native application, and how they are connected. With inspiration from [5].

Host Component to be mounted, and JavaScript information about props. They also contain layout information such as coordinates and size (width and height). React Native executes the same React framework code as React for the web. But React for the web renders DOM nodes (web's host views). React Native instead renders what is called host views instead of DOM nodes. Host views are in other terms, general platform views. The Fabric Renderer is tasked with rendering these host views. The Fabric Renderer lets React talk to each platform and manage its host view instances. It exists in JavaScript and targets interfaces that are made available by C++ code.

Some popular apps built with the React Native technology are Discord, Coinbase, and Shopify, according to [8].

3 Methods

In this study mainly two research methods has been used. The first is an analysis of the structure of Ionic and React Native. This analysis has investigated the two frameworks different approaches for creating cross-platform applications. The second is a survey to get subjective and quantitative data from developers regarding their opinions towards Ionic and React Native.

3.1 Analysis

The analysis of this study aim to support to findings of the online survey. The documentation that has been looked at are articles found online from authors with experience in cross-platform frameworks. These sources discuss the differences between Ionic and React Native. The frameworks have been compared by how they can be used and what limitations they have from a developer perspective.

3.2 Survey

The second research method was an online survey. The survey that was created for this study consisted of questions related to Ionic and React Native from a developer perspective. Naturally this survey had to be taken by people with experience in Ionic and/or React Native. The survey was created by using the survey creator in Microsoft Forms [6]. Microsoft Forms has also been used to gather and export the total survey data from the participants. The survey has followed the Umeå University survey guidelines, as well as their consent form to process personal data which can be found in [17]. Before the official survey was published a pilot survey was performed. This pilot study was tested on students at Umeå University and some of the survey questions changed based on the students feedback of this pilot survey. The data gathering and processing was performed during November of 2025.

3.2.1 Participants. For the sample group of this survey a strict random sampling can not be done. Therefore a non-probability sample has been used, explained in [11]. This survey was published in online communities where developers with potential experience in the relevant frameworks could be found. A participant would be eligible to take part in the survey if they had experience in React Native or Ionic, or both. The content of the survey was the same for all participants, no matter the individual participant experience.

The online communities that the survey was published on are all found on Reddit [16]. On Reddit the survey was posted in the following communities: r/iOSDevelopment, r/developer, r/appdev, r/AppDevelopers, r/reactjs, r/react, r/ionic, r/reactnative, and r/MobileAppDevelopers. From these communities a total of 15 users took the survey. All of these participants were completely anonymous. It was also unknown which community each participant belonged to.

4 Result

This section of the paper is divided into two sections. The first being the results of the online analysis on the two frameworks. The later being the quantitative results from the online survey.

4.1 Analysis Results

In an article from GeeksforGeeks in 2023 [7] the differences between Ionic and React Native are discussed. The first interesting difference that this article mentions is regarding the frameworks differences in rendering efficiency. The article states that React Native is faster than Ionic. The article claims that this is because of Ionic's architecture and Cordova, which renders the application as a web app. As was described in subsection 2.2 of this paper. This article thus states that rendering applications as web apps is slower than rendering them natively as React Native does. However this could also be a positive thing for developing in Ionic. Since Ionic applications can be tested in any web browser. Meanwhile if a developer properly wants to test and debug a React Native application, a mobile device has to be used. Cordova is also what Ionic uses to access the hardware of a device. React Native does not need any external support for accessing hardware.

Next the article briefly mentions the documentation of these frameworks. According to the article Ionic's documentation is simple, clear and consistent compared the React Native's. The article concludes its findings as such *"Ionic is a better choice if you want to build applications quickly and if you have web development skills. React Native is a better choice if you want a native look and feel and if you prioritize performance. Ultimately, the choice between Ionic and React Native depends on your specific needs and the requirements of your project."*

In an article published on Ionic's website Max Lynch also writes about the differences in Ionic React and React Native [13]. Ionic React is a term for using the Ionic framework with React integrated in the codebase, Ionic React is the official React version of the Ionic framework. This article also compares the rendering approaches of Ionic (with React) and React Native. It describes Ionic React as web-first, meaning that it implements native iOS and Android UI using cross-platform, standards-based web technology instead of accessing platform UI controls directly. In contrast, React Native provides an abstraction through React that can control platform UI controls directly. Due to this approach by React Native its performance may be higher and it is easier to interface with an existing native apps UI controls, according to the article. However, the article states, this approach does have some downsides. Native controls are harder to customize than what most web developers and designers might be used to. And since React Native is not a true browser environment, standard CSS, HTML, and DOM techniques can not be used, unlike Ionic React. Although React Native does have a CSS-like utility.

Ionic React has Progressive Web App support and React Native does officially not support it. This also means that the development process for each option is different. Ionic React is very similar to traditional React web app development. That means Ionic React can offer a web developer a fast development experience directly in a browser of their choice, and many apps can have a significant portion of their functionality built right in the browser.

The article explains the Ionic React approach as "write once, run anywhere". And React Native's approach as "learn once, write anywhere". With React Native, developers learn one set of concepts, and then build UI screens specifically for iOS and specifically for Android. In contrast, Ionic React apps run the same UI on all platforms, utilizing responsive web design, and CSS to enable developers to customize an app for specific platforms if they choose.

The article concludes by discussing which option is better depending on the developer team project. For teams targeting iOS and Android only and with more traditional native developers or advanced JavaScript developers and an existing repository of native controls, React Native may be a better fit. This explains why React Native is popular with consumer app startups that have a history of native app development. For teams that have traditional web development skills and libraries and wish to target mobile and web (as a Progressive Web App), Ionic React will likely be a better fit. This explains why Ionic has been successful with startups and enterprise teams with more of a web development history. Additionally, for teams that need guaranteed support and dedicated enterprise functionality, Ionic React may be the only option, given that it is one of few platform companies in this space dedicated to the business of app development technology.

4.2 Survey Results

Among a total of 15 participants 14 consented to take part in survey and share their data with Umeå University for research purposes. When asked about their primary roles two of these participants answered “Front-end developer”, three said “Mobile-app developer”, and nine said “Full-stack developer”.

Out of the 14 participants seven claimed to have more than eight years of experience in mobile development. Four participants said they have somewhere between five and seven years of experience. The participants were then asked about their experience with various frameworks. For this question participants could fill in multiple answers. All 14 participants claimed to have experience in Ionic, and have used it in a production app. Nine participants claimed to have experience in React Native, but in that group only eight of them had used React Native in a production app. Out of the total sample group only three stated to have experience in frameworks other than Ionic and React Native. These free-text answers consisted of Flutter, Xamarin and iOS/Android Native.

Since all participants have claimed to have experience in Ionic, but not all in React Native, the results of the survey has thus been divided into two groups. The first group consists of nine participants. These are the participants with both React Native, and Ionic experience. The other group consists of the remaining participants, five in total. These have all used Ionic and possibly some other frameworks as well. What this group have answered for in regards to React Native has not be taken into account in this section or the coming sections of this paper. Since this group does not have any experience of using React Native.

The group with experience in both React Native and Ionic has been given the label **RIU**, short for React Native and Ionic Users. The remaining participants that only had experience with Ionic has been given the label **IU**, short for Ionic Users. To make the following result tables fit inside this article columns, React Native has been abbreviated to RN. The results for each question appear in the same order that the survey had.

4.2.1 General performance. Participants were asked: “How would you rate the general performance of these frameworks?”. When the participants were asked about the performance of the each framework, there was no unit specified. This measurement was simply based on each participant’s personal experience of performance. The results are displayed in Table 1.

Table 1: Ratings for general performance of each framework.

Q1	RIU for RN	RIU for Ionic	IU for Ionic
Excellent	5	0	1
Good	3	5	0
Average	0	0	0
Fair	1	4	4
Poor	0	0	0

4.2.2 Development efficiency. Participants were asked: “Which framework allows you to develop faster?”. The results are displayed in Table 2.

Table 2: Preferences in development efficiency.

Q2	RIU	IU
React Native	4	0
Ionic	4	4
About the same	1	1
Not sure	0	0

4.2.3 Learning difficulty. Participants were asked: “How difficult was it to learn these frameworks?”. The results are displayed in Table 3.

Table 3: Ratings for learning difficulty of each framework.

Q3	RIU for RN	RIU for Ionic	IU for Ionic
Very difficult	0	0	0
Difficult	4	0	0
Neutral	2	2	1
Easy	3	5	3
Very easy	0	2	1

4.2.4 Looks and feel of applications. Participants were asked: “Which framework excelled in creating better-looking and more “native-feeling” apps?”. The results are displayed in Table 4.

Table 4: Preferences in better-looking and native-feeling applications.

Q4	RIU	IU
React Native	7	2
Ionic	0	1
About the same	2	2
Not sure	0	0

4.2.5 Plugins and libraries. Participants were asked: “Which framework has better plugins and libraries available for your needs?”. The results are displayed in Table 5.

Table 5: Preferences in plugins and libraries.

Q5	RIU	IU
React Native	6	0
Ionic	1	2
About the same	2	1
Not sure	0	2

4.2.6 Documentation quality. Participants were asked to: “Rate the documentation quality for these frameworks.”. The results are displayed in Table 6.

Table 6: Ratings for documentation quality of each framework.

Q6	RIU for RN	RIU for Ionic	IU for Ionic
Excellent	2	4	2
Good	3	3	2
Average	3	1	0
Fair	1	1	1
Poor	0	0	0

4.2.7 *Cross-platform consistency.* Participants were asked: “Which framework provides better cross-platform consistency?”. The results are displayed in Table 7.

Table 7: Preferences in cross-platform consistency.

Q7	RIU	IU
React Native	4	1
Ionic	3	2
About the same	2	1
Not sure	0	1

4.2.8 *Future projects.* Participants were asked: “Which framework would you chose for your next project?”. The results are displayed in Table 8.

Table 8: Preferences for which framework to choose in the future.

Q8	RIU	IU
React Native	6	0
Ionic	1	3
Depends on the project	1	2
Not sure	1	0

4.2.9 *Overall satisfaction.* Participants were asked to: “Rate your overall satisfaction of these frameworks.”. The results are displayed in Table 9.

Table 9: Ratings for overall satisfaction of each framework.

Q9	RIU for RN	RIU for Ionic	IU for Ionic
Excellent	4	2	3
Good	4	3	1
Average	1	1	1
Fair	0	1	0
Poor	0	2	0

4.2.10 *Free-text.* Participants were asked: “What was the main reason you chose one framework over the other?”. This question of the survey was a free-text answer. Out of all the participants eleven submitted an answer for this last question. The other three did not submit anything here. These eleven participants provided the following free-text responses:

- “Because i am Vue developer I prefer ionic” - *Participant ID 6*
- “Easier to learn. Even if I didn’t like it I’d still have web dev skills” - *Participant ID 7*
- “I haven’t worked with React Native yet, but I’m learning React so I can use it in the future. I’ve worked with Ionic on a production app because it was an existing app already in production.” - *Participant ID 8*
- “1.) Market dynamics : - React native has more jobs compared to Ionic. 2.) Lack of cordova support: - Ionic was pleasure to work when they were built on top of cordova/phonegap. This allowed lot of enterprise plugins to work on the project. After driftco moved to capacitor, it cause lot of conflicts. The current cordova support is not as great as say Ionic v4” - *Participant ID 5*
- “Easier to write plugins for ionic rather than react native.” - *Participant ID 9*
- “Ionic just sucks. It’s like Xamarin. Find me one dev that has used Ionic, tried RN or kotlin multiplatform or anything else and would switch back to Ionic. Yeah you will find almost nobody just like for Xamarin/Maui” - *Participant ID 10*
- “A react native app creates actual native view elements and at the end of the day it is a native app experience. An ionic app is a web page in an app shell with native look, but not feel. React Native is a more mature experience and using tools like expo modules you can easily create native integrations for any potentially missing functionality and plug it directly into your app. Ionic is a stop gap for making your mobile website act like an app, React native is a true mobile app” - *Participant ID 11*
- “Balance between development speed and overall application quality.” - *Participant ID 13*
- “ionic was more ready for production first” - *Participant ID 15*
- “Project type” - *Participant ID 16*
- “Performance is main key here Compare to ionic Reaction native is high performance and better rendering and native feel” - *Participant ID 17*

5 Discussion

This section of the paper is divided into two sections. The first being a discussion around the findings from the analysis of articles. The later being a discussion around the survey results.

5.1 Analysis Discussion

The two articles found during the analysis mention a lot of the same topics. The topics being rendering, structure, usability for developers, and performance. Something worth mentioning is that the article found on Ionic’s website [13] is written by an employee of Ionic, Max Lynch. Which would explain why the article seems to favour Ionic (with React).

One thing that the article on GeeksforGeeks [7] mentioned that [13] did not, is documentation quality. The GeeksforGeeks article clearly stated that Ionic’s documentation is of higher quality than React Native. Both articles talk a lot about development efficiency and native capabilities. The articles agree that Ionic excels in simplicity for creating cross-platform applications. Since Ionic

can render the same application for different devices given the web app functionality given by Cordova. The articles also (somewhat) agree that React Native is the superior in creating applications with native components.

The article by Lynch [13] is the only article that talks about which framework developer teams should chose. Which is very relevant for the objective of this paper. According to Lynch the choice should be based on what the existing repository consists of, and what the project goal is. If a team wants to target iOS and Android and has a repository of native controls, React Native is the better choice. If a developer team wants to target mobile and web and has *traditional web development skills*, Ionic would be more suitable. So in short if the project has a history of native app development, chose React Native. If the project has a history of web development, chose Ionic.

5.2 Survey Discussion

From the survey data a majority of the participants were full-stack developers, nine in total. The other part of the participants consisted of front-end and mobile-app developers, five in total. These are all very relevant roles for developing mobile applications. Most of these participants had quite a lot of experience in mobile development. A majority of eight people had more than eight years of experience and the rest had between five and one year/s of experience.

All of these participants have claimed to have some experience with Ionic and also having used it in a production app. This could likely be due to the Reddit communities the survey was posted on. One of which was the community *r/ionic*. This community had the least amount of members out of the communities that the survey was posted on. It is thus likely that most participants came from this community. Since that particular survey post was less likely to be overshadowed by other posts in *r/ionic*. Meaning that it was more likely for members of that community to actually see the post in their feed.

When we analyse the survey results further we still refer to the same groups that were used in subsection 4.2. For some questions on the survey the total amount of participant selections has been discussed, but not for all questions. Since in questions were the participant chooses between frameworks in their answer, it is for us only relevant to look at the **RIU** group.

For Question 1 in subsection 4.2.1, regarding general performance, no one in the **RIU** group said that Ionic's performance is excellent. But five in group said that React Native's general performance is excellent. The one participant that chose 'Excellent' for Ionic only had experience in Ionic. Among the **RIU** group it is apparent that React Native is favoured. This result could potentially be correlated with the two frameworks rendering methods. As was discussed in the articles [7] and [13]. Where it is mentioned that typically a web app has a lower performance.

In Question 2 regarding learning difficulty, in subsection 4.2.3, it is apparent that **RIU** thinks React Native is more difficult to learn than Ionic. If we look at all the participants they all rated Ionic's learning difficulty as neutral or even less difficult than that. This result corresponds well with the findings in the article [7], if we take documentation quality into consideration here. It is also hard to compare the participants learning experience since we can not know what experience they had beforehand. For example it would

be easier to learn React Native if you have had previous experience with React.

For Question 4 regarding looks and feel of applications, in subsection 4.2.4. The question can be a bit diffuse and participant could interpret this for different things, which would have influenced the results of course. This is also a question where we only want to look at the **RIU** group. In this group 77% preferred the looks of React Native and the rest thought of it as not too different. Apart from the one that preferred Ionic. From the **RIU** group we can see that generally React Native is the preferred option when it comes to looks and feel of an application. It is possible that this is also because of Ionic's and React Native's different rendering methods. As mentioned in article [13].

For Question 6 regarding documentation quality in subsection 4.2.6. A majority of **RIU** rated React Native's documentation as good or excellent. Among all the participants a majority rated Ionic's documentation as good or excellent. If we look at percentages for these ratings we can see that React Native got 22% for "Excellent" and 33% for "Good". For Ionic 42% rated the documentation quality as "Excellent" and 35% for "Good". We can then see that based on percentages Ionic got a higher rating in documentation quality. This also corresponds well with what was stated in article [7]. Where the article described Ionic's documentation as simple, clear and consistent. Though the article did not mention anything in particular as to what makes React Native's documentation worse than Ionic's.

For Question 7 regarding cross-platform consistency in subsection 4.2.7. In the **RIU** group the React Native option had the most individual selections. Although the options Ionic and 'About the same' had a majority in total within this group. In the **IU** group the answers were very spread. We can see that in the **RIU** group React Native is the preferred option, by a hair. This could also be connected to the findings in article [13]. Where it was mentioned that React Native has the ability to control the platform UI controls directly. Which would make the application feel more native. In comparison to how Ionic does this by using a web-first approach.

In Question 8 regarding what framework they would chose for their next project, in subsection 4.2.8. In the **RIU** group 66% said that they would chose React Native, and 11% said they would chose Ionic. For the **IU** group a majority chose Ionic for their next project. We can conclude that developers with experience in both frameworks would most likely choose React Native for their next project. And those with only Ionic experience would most likely chose Ionic in the future.

Almost all of the participants (11 out of 14) had something to say in the free-text field in subsection 4.2.10. Some of the more interesting submissions were from participant ID 5 and ID 11. Participant ID 5 talks about how Ionic used to be great and work well with Cordova. But since Drifty Co. moved to capacitor, the support for Cordova has decreased.

Participant ID 11 talks a lot about the core differences of an Ionic and React Native application. ID 11 mentions a lot of the same points that were also brought up in the articles [13] and [7]. This participant talks about how React Native creates true native elements for a better app experience. Meanwhile Ionic renders an app in the form of a web page in an app shell that looks native, but really it is not.

6 Conclusions

There are a couple of interesting findings in the articles that can support some of the survey results. We can see that for documentation quality Ionic is better than React Native. Both according to articles and the experienced developers that took the survey. The survey results also show that React Native generally has a better performance than Ionic. Both in looks/feel and just performance in general. But it is a little difficult to make any connections to the articles for these results in the survey. Since the performance metric in the survey was not specified at all and it was up to the participants to interpret it. Although React Native was favoured in both general performance and looks/feel. We can however make the conclusion that React Native has a better native compatibility and feels more like a real native mobile application. The results for this is clearly reflected in both the articles and the survey.

The only source that specifically gives guidelines in which framework to choose is the article from Max Lynch [13]. His article stated that a developer team with a history of native app development, should choose React Native. A team with history of web development, should choose Ionic. Since this statement is only claimed in this one article, and none of the survey results support this, there can be no conclusion made for this statement.

Based on the results from the articles, and the survey results that support each other, there are a few things we can say regarding which framework to choose. If you want to learn a cross-platform framework quickly, deploy it across multiple devices under a short time, Ionic is the framework you should choose. But if you prioritize looks and feel, native capability especially on mobile devices, React Native is the way to go. But since the number of survey participants was too low there can be no scientifically valid conclusions made for this study.

Acknowledgements

Notebooklm was utilized to correct spelling and grammar mistakes in this paper.

References

- [1] Apache. 2025. *Documentation Overview*. Retrieved November 21, 2025 from <https://cordova.apache.org/docs/en/3.0.0/guide/overview/>
- [2] Nicola Corti. 2024. *Use a framework to build React Native apps*. Retrieved December 8, 2025 from <https://reactnative.dev/blog/2024/06/25/use-a-framework-to-build-react-native-apps>
- [3] React Native devs. 2024. *Glossary*. Retrieved November 21, 2025 from <https://reactnative.dev/architecture/glossary>
- [4] React Native devs. 2025. *Get Started with React Native*. Retrieved December 8, 2025 from <https://reactnative.dev/docs/environment-setup>
- [5] Rahul Dubey. 2022. *React Native Architecture*. Retrieved October 29, 2025 from <https://www.educba.com/react-native-architecture/>
- [6] Microsoft Forms. 2025. *Homepage of Microsoft Forms*. Retrieved November 21, 2025 from <https://www.microsoft.com/en-us/microsoft-365/online-surveys-polls-quizzes>
- [7] Geeksforgeeks. 2023. *Difference between Ionic and React Native*. Retrieved January 3, 2026 from <https://www.geeksforgeeks.org/reactjs/difference-between-ionic-and-react-native/>
- [8] Fatos Morina Gergely Orosz, Elin Nilsson. 2025. *Cross-platform mobile development*. Retrieved November 3, 2025 from <https://newsletter.pragmaticengineer.com/p/cross-platform-mobile-development>
- [9] Infostride. 2025. *Top 5 Popular Apps Built with Ionic Framework*. Retrieved November 3, 2025 from <https://infostride.com/apps-built-with-ionic/>
- [10] Ionic. 2025. *Ionic Open Source Documentation*. <https://ionicframework.com/docs>
- [11] Harry Hochheiser Jonathan Lazar, Jinjuan Heidi Feng. 2017. *Research Methods in Human-Computer Interaction* (2nd ed.). Vol. 1. Morgan Kaufmann Publishers, Cambridge, MA, USA.
- [12] Ihor Lobastov. 2019. *The Good and the Bad of Ionic Mobile Development*. Retrieved October 29, 2025 from <https://dzone.com/articles/the-good-and-the-bad-of-ionic-mobile-development>
- [13] Max Lynch. 2025. *Ionic React vs React Native: What are the Differences Between Them?* Retrieved January 3, 2026 from <https://ionic.io/resources/articles/ionic-react-vs-react-native>
- [14] JetBrains Modifiers. 2025. *The Six Most Popular Cross-Platform App Development Frameworks*. Retrieved October 29, 2025 from <https://www.jetbrains.com/help/kotlin-multiplatform-dev/cross-platform-frameworks.html>
- [15] React Native. 2025. *React Native Open Source Documentation*. <https://reactnative.dev/docs/getting-started>
- [16] Reddit. 2025. *Homepage of Reddit*. Retrieved November 21, 2025 from <https://www.reddit.com/>
- [17] Umeå University. 2025. *Processing of personal data*. Retrieved November 21, 2025 from <https://www.umu.se/en/about-the-website/legal-information/processing-of-personal-data/>
- [18] Business Wire. 2022. *OutSystems Acquires Ionic to Unite the Power of Open Source Mobile Development with High-Performance Low-Code*. Retrieved November 3, 2025 from <https://www.businesswire.com/news/home/20221107005221/en/OutSystems-Acquires-Ionic-to-Unite-the-Power-of-Open-Source-Mobile-Development-with-High-Performance-Low-Code>